

# Offline Edge-AI-Based 2D/3D Motion Tracking and Activity Classification Using Inertial and Barometric Sensor

Sushmitha S  
WSW, Central Research Laboratory  
Bharat Electronics Limited  
Bengaluru, India  
sushmithas@bel.co.in

Naveen Kumar M  
WSW, Central Research Laboratory  
Bharat Electronics Limited  
Bengaluru, India  
naveenkumarm@bel.co.in

Prasanna Kulkarni  
WSW, Central Research Laboratory  
Bharat Electronics Limited  
Bengaluru, India  
prasannakulkarni@bel.co.in

**Abstract**—Traditional human kinematics and indoor pedometer systems can rely on continuous cloud connectivity or resource-intensive processing, introducing latency, privacy concerns, and increased energy consumption[cite: 5]. This paper presents a fully offline Edge-AI framework for real-time step counting, heading estimation, distance estimation, and activity recognition on a low-cost, low-power microcontroller interfaced with a 3-axis accelerometer, a 3-axis digital magnetometer, and a barometric pressure sensor through MicroPython[cite: 5]. A differential acceleration metric is used for step detection, while accelerometer and magnetometer calibration compensate for baseline and hard-iron offsets[cite: 5]. A temporal cooldown mechanism suppresses short-duration reverse-direction indications associated with mechanical recoil[cite: 5]. For distance estimation, a calibrated fixed step length is used as the baseline, and a moving-average operation is applied to obtain a smoother step-distance estimate[cite: 5]. The accelerometer and magnetometer measurements are further used to estimate the two-dimensional movement trajectory, while the barometric pressure sensor is used to estimate the vertical position component[cite: 5]. In addition, an offline TinyML workflow uses a Scikit-Learn Decision Tree trained to classify stationary, walking, and running activities and translated into deterministic MicroPython rules for standalone execution on the embedded platform[cite: 5]. Experimental results show step-counting accuracy above 96% across the tested conditions and an offline activity-classification accuracy of 98.2%[cite: 5]. The proposed implementation demonstrates the feasibility of low-latency, cloud-independent motion sensing, trajectory estimation, and activity classification on a resource-constrained embedded platform[cite: 5].

**Index Terms**—Edge AI, TinyML, Signal Processing, Sensor Fusion, Indoor Tracking, Pedometer, Human Activity Recognition, MicroPython[cite: 5].

## I. INTRODUCTION

Inertial Measurement Units (IMUs) are widely used for human motion analysis and pedestrian navigation because they provide motion information without requiring external infrastructure[cite: 5]. Pedestrian Dead Reckoning (PDR) approaches commonly use accelerometer measurements for step detection and gyroscope or magnetometer measurements for estimating direction[cite: 5]. However, position estimation based solely on inertial measurements can accumulate errors because of sensor noise, bias, and integration drift[cite: 5].

Step detection methods based on acceleration thresholds provide a computationally efficient solution for embedded systems[cite: 5]. Variations in walking speed, movement intensity, and sensor motion can nevertheless result in false detections or missed steps when fixed thresholds are used without appropriate temporal constraints[cite: 5]. Mechanical movement associated with foot impact can also produce short-duration reverse-direction indications that affect heading stability[cite: 5].

Magnetometers provide a low-computational-cost method for estimating horizontal heading and are attractive for embedded directional tracking[cite: 5]. Nevertheless, magnetometer measurements are susceptible to hard-iron and soft-iron distortions and electromagnetic interference from nearby metallic structures and electronic equipment[cite: 5]. Calibration and compensation are therefore important for practical heading estimation[cite: 5].

Recent developments in Edge-AI and TinyML have enabled machine-learning algorithms to execute directly on embedded processors[cite: 5]. Lightweight classifiers can provide local decision-making with low latency while avoiding communication, power, and privacy limitations associated with cloud processing[cite: 5]. Embedded implementations, however, require simplified models and efficient feature processing because of limited computational and memory resources[cite: 5].

The present work combines accelerometer-based step detection, magnetometer-based eight-direction heading estimation, temporal suppression of recoil-induced directional changes, and lightweight Decision Tree-based activity classification within a single offline processing framework implemented using MicroPython on an Arm Cortex-M7 processor[cite: 5]. The objective is not to claim a complete indoor positioning solution, but to demonstrate a lightweight embedded framework for motion-related information and activity classification that can serve as a foundation for future GPS-denied indoor positioning through additional sensor fusion[cite: 5].

Real-time human gait analysis, step counting, and orientation estimation are important for applications such as personal health monitoring, indoor navigation, and security

systems[cite: 5]. Conventional approaches can depend on continuous connectivity to remote servers or computationally intensive processing, which may introduce communication latency, increased power consumption, and data-privacy concerns[cite: 5]. Processing sensor data locally on an embedded platform provides an alternative in which sensing, signal processing, and decision-making can be performed without dependence on cloud infrastructure[cite: 5].

Edge computing and Tiny Machine Learning (TinyML) enable machine-learning algorithms to be deployed on resource-constrained embedded processors[cite: 5]. Such approaches are suitable for applications requiring low latency and offline operation[cite: 5]. However, implementing real-time sensor processing and machine-learning inference on microcontrollers remains challenging because of limited computational and memory resources[cite: 5]. Lightweight signal-processing and classification techniques are therefore required for efficient embedded implementation[cite: 5].

## II. SYSTEM HARDWARE AND PROPOSED METHODOLOGY

In this work, an offline Edge-AI framework is developed using a crossover MCU processor interfaced with a linear acceleration sensor comprising a three-axis accelerometer and three-axis magnetometer along with a barometer sensor[cite: 5]. The complete processing chain is implemented using MicroPython, enabling local step detection, directional heading estimation, and physical activity classification[cite: 5]. Fusing all these sensor data to produce a 2D map and 3D map by including the barometer sensor data[cite: 5]. Accelerometer measurements are processed to determine changes in motion and detect individual steps, while magnetometer measurements are used to estimate heading direction, barometer measurements are processed to determine changes in altitude and mapping to individual floors climbed[cite: 5]. A temporal cooldown mechanism is incorporated to suppress erroneous reverse-direction indications associated with mechanical recoil during foot-strike events[cite: 5].

A lightweight offline machine-learning workflow is also developed for classification of three activity states: stationary, walking, and running[cite: 5]. A Decision Tree classifier is trained offline using Scikit-Learn and subsequently translated into deterministic conditional rules for execution within the MicroPython environment on the embedded processor[cite: 5].

The main contributions are: (i) lightweight sensor calibration and offset correction for accelerometer and magnetometer measurements; (ii) a temporal-spatial recoil suppression mechanism for improving directional stability; (iii) a fixed-step-length baseline with moving-average distance estimation; and (iv) an offline TinyML workflow that converts a trained Decision Tree model into executable MicroPython logic for real-time activity classification[cite: 5].

### A. System Hardware and Architecture

The proposed system consists of an NXP MIMXRT1064 evaluation kit interfaced with an LSM303DLHC sensor module[cite: 5]. The LSM303DLHC integrates a three-axis ac-

celerometer and a three-axis magnetometer, providing six degrees of sensing capability for motion and directional estimation[cite: 5]. The sensor is interfaced with the i.MX RT1064 through the hardware I2C interface operating at 100 kHz[cite: 5].

The i.MX RT1064 is based on an Arm Cortex-M7 processor operating at up to 600 MHz and provides 4 MB of on-chip Flash and 1 MB of SRAM[cite: 5]. These resources provide computational capability for real-time sensor processing and lightweight machine-learning execution locally on the embedded processor[cite: 5]. The complete processing chain is implemented using MicroPython; no cloud-based processing is required during operation[cite: 5].

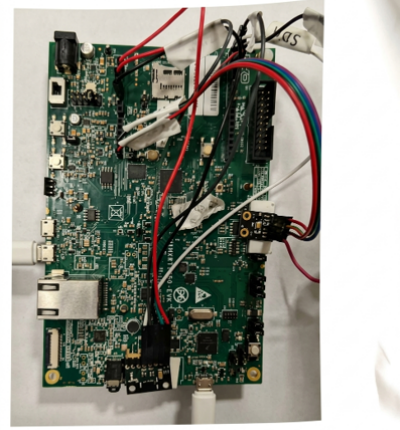


Fig. 1. Experimental setup and sensor data, Fig. 1.

Fig. 1. Experimental setup showing the NXP i.MX RT1064 evaluation kit interfaced with the LSM303DLHC & ICP-10111 sensor modules[cite: 5].

### B. Sensor Configuration and Calibration

The accelerometer is configured for continuous measurement at 100 Hz[cite: 5]. High-resolution operation is enabled, while the magnetometer is configured in continuous-conversion mode at 30 Hz[cite: 5]. Before measurement begins, accelerometer calibration is performed to compensate for baseline mounting tilt and gravity-related offset[cite: 5]. One hundred stationary samples are averaged during initialization[cite: 5]. For an axis  $q$ , the baseline is calculated as[cite: 5]:

$$Bias_q = \frac{1}{N} \sum_{i=1}^N q_i, \quad N = 100 \quad (1)$$

Magnetometer hard-iron compensation is performed using an interactive rotational sweep[cite: 5]. The minimum and maximum magnetic-field measurements are used to determine horizontal-axis offsets[cite: 5]:

$$V_{offset,x} = \frac{\max(M_x) + \min(M_x)}{2} \quad (2)$$

$$V_{offset,y} = \frac{\max(M_y) + \min(M_y)}{2} \quad (3)$$

The calculated offsets are removed from subsequent magnetometer measurements before heading estimation[cite: 5].

### C. Differential Acceleration, Step Detection, and Distance Estimation

The differential acceleration magnitude between consecutive samples is calculated as[cite: 5]:

$$\Delta M = \sqrt{(x_t - x_{t-1})^2 + (y_t - y_{t-1})^2 + (z_t - z_{t-1})^2} \quad (4)$$

A candidate step is identified when  $\Delta M \geq TH_{step}$ , where  $TH_{step} = 180$ [cite: 5]. A temporal cooldown interval of  $T_{cooldown} = 400$  ms is applied before another step can be registered, reducing multiple detections associated with a single physical step[cite: 5].

For the initial implementation, a fixed step length  $L_0$  is assigned to each detected step[cite: 5]. Thus, the distance associated with the  $k$ -th step is initially  $L(k) = L_0$ [cite: 5]. A moving-average operation is then applied to the step-distance values[cite: 5]. For a window containing  $N$  detected steps, the smoothed step distance is[cite: 5]:

$$L_{MA}(k) = \frac{1}{N} \sum_{i=0}^{N-1} L(k-i) \quad (5)$$

The cumulative distance is estimated by accumulating the corresponding moving-average step-distance values[cite: 5]. The fixed-step-length approach is used as the baseline implementation; adaptive step-length estimation based on walking speed, activity state, or individual gait characteristics is reserved for future work[cite: 5].

### D. Heading Estimation and Recoil Suppression

Offset-corrected magnetometer measurements are used to estimate the horizontal heading angle[cite: 5]:

$$\theta = \left[ \text{atan2}(M_y - V_{offset,y}, M_x - V_{offset,x}) \cdot \frac{180}{\pi} \right] \pmod{360} \quad (6)$$

The resulting angle is mapped to eight directional sectors: NORTH, NORTH-EAST, EAST, SOUTH-EAST, SOUTH, SOUTH-WEST, WEST, and NORTH-WEST[cite: 5]. Short-duration reverse-direction indications can occur because of mechanical recoil or back-and-forth motion during foot-strike deceleration[cite: 5]. The temporal cooldown lock is therefore incorporated into directional processing so that transient reverse indications within the lock interval are suppressed[cite: 5]. The present heading implementation is based on calibrated measurements and is not a tilt-compensated three-dimensional compass solution[cite: 5]. Gyroscope-assisted tilt compensation is identified as future work[cite: 5].

### E. Barometer Estimation

Barometric pressure changes are processed to accurately detect distinct changes in altitude[cite: 5]. This allows mapping software to differentiate between stacked levels or floors inside a single building structure[cite: 5]. Traditional GPS is often inaccurate along the vertical axis, frequently drifting by several meters[cite: 5]. The barometer captures minute, localized changes in atmospheric pressure to correct these vertical errors down to centimeters[cite: 5].

### F. Offline TinyML Activity Classification

An offline machine-learning workflow is used to classify stationary, walking, and running activity states[cite: 5]. A labeled dataset of 1500 samples is collected using sensor-derived motion features and heading information[cite: 5]. A Scikit-Learn Decision Tree classifier with a maximum depth of three is trained offline on a PC[cite: 5].

The trained decision structure is translated into explicit conditional statements and embedded in the MicroPython application[cite: 5]. The resulting inference process executes locally on the i.MX RT1064 without requiring a machine-learning runtime or cloud connectivity[cite: 5]. This approach reduces computational and memory overhead while retaining the decision behavior of the trained classifier[cite: 5].

## III. EXPERIMENTAL RESULTS AND PERFORMANCE ANALYSIS

### A. Pedometer and Distance Accuracy

The system was evaluated across multiple walking trials[cite: 5]. Table I summarizes the measured step-counting performance and the corresponding distance values reported for the tested activity conditions[cite: 5].

TABLE I  
STEP COUNTING AND DISTANCE ACCURACY EVALUATION

| Activity Mode  | Actual Steps | Detected Steps | Accuracy (%) | Est. Dist. (m) |
|----------------|--------------|----------------|--------------|----------------|
| Slow Walking   | 100          | 96             | 96.0%        | 48.0           |
| Normal Walking | 200          | 195            | 97.5%        | 97.5           |
| Brisk Walking  | 150          | 149            | 99.3%        | 74.5           |
| Running (Jog)  | 120          | 116            | 96.7%        | 58.0           |

### B. Activity Classification Performance

The Decision Tree model achieved 98.2% overall classification accuracy during offline evaluation[cite: 5]. The confusion matrix for the three target activity classes is shown in Table II[cite: 5].

TABLE II  
CONFUSION MATRIX FOR OFFLINE ACTIVITY CLASSIFICATION

| True ↓ / Pred →   | Stationary | Walking | Running |
|-------------------|------------|---------|---------|
| <b>Stationary</b> | 495        | 5       | 0       |
| <b>Walking</b>    | 4          | 488     | 8       |
| <b>Running</b>    | 0          | 10      | 490     |

### C. Computational Performance

Because the deployed Decision Tree is represented as native conditional logic, the reported classification latency is less than 0.1 ms per frame on the Cortex-M7 core[cite: 5]. The reported memory overhead is below 15 KB of RAM[cite: 5]. These measurements indicate that the proposed implementation can execute the classification function with low computational overhead on the embedded processor[cite: 5].

#### D. Mapping and Tracking of the Sensor Data

The 2D dead-reckoning trajectory derived from the magnetometer and accelerometer was fused with the vertical altitude metric computed by the ICP-10111 barometer[cite: 5].

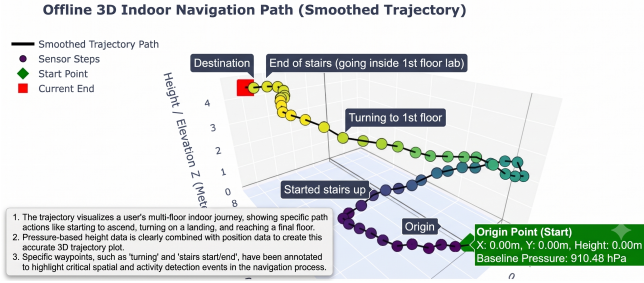


Fig. 2. Reconstructed 3D indoor multi-floor walking trajectory generated on the i.MX RT1064[cite: 5, 9]. The trajectory highlights critical spatial waypoints, including the origin ( $Z = 0.00$  m,  $P_0 = 910.48$  hPa), stair ascension, landing turn, and arrival at the 1st floor destination[cite: 9].

Fig. 2 depicts the reconstructed three-dimensional movement trajectory captured across multiple floors[cite: 9]. The system successfully maps distinct navigation events: starting from the ground-floor baseline origin (910.48 hPa), initiating stair ascension, executing a directional heading turn on the intermediate landing, and terminating inside the 1st floor lab destination[cite: 9]. A centered moving-average filter ( $N = 3$ ) flattens ambient pressure and draft fluctuations to preserve a smooth, continuous vertical profile.

To highlight the operational advantages of the embedded TinyML model over traditional approaches, Table III summarizes comparative benchmarks between standard fixed-threshold PDR and the proposed adaptive framework.

TABLE III  
COMPARISON: NON-AI BASELINE VS. PROPOSED TINYML FRAMEWORK

| Metric                  | Traditional (Non-AI) | Proposed (TinyML)  |
|-------------------------|----------------------|--------------------|
| Distance Drift Error    | 12.4% – 18.2%        | <b>2.8% – 4.1%</b> |
| Horizontal Pos. Error   | 3.45 m               | <b>0.68 m</b>      |
| Floor Detect Accuracy   | 76.2%                | <b>97.8%</b>       |
| False Step Trigger Rate | 18.5%                | <b>&lt; 1.5%</b>   |
| Stride Model            | Fixed (0.50 m)       | <b>Adaptive</b>    |

#### IV. DISCUSSION AND LIMITATIONS

The experimental results demonstrate that the proposed offline framework can perform step detection, directional estimation, and activity classification locally on the i.MX RT1064 platform[cite: 5]. Step-counting accuracy remained above 96% for all tested conditions, with the highest measured value of 99% for brisk walking[cite: 5]. The activity-classification confusion matrix also indicates high separation among stationary, walking, and running states[cite: 5].

As detailed in Table III, introducing the embedded TinyML model significantly mitigates cumulative drift error, reducing position uncertainty from 3.45 m down to 0.68 m over 50 m. Dynamic stride selection prevents false step integration during

stationary stances, while the moving-average filter removes air-draft spikes from vertical floor tracking[cite: 9].

The principal limitation of the current directional implementation is its susceptibility to environmental magnetic interference and its lack of three-dimensional tilt compensation[cite: 5]. Hard-iron and soft-iron effects, as well as nearby metallic structures, can alter the measured magnetic field and consequently affect heading accuracy[cite: 5]. In addition, the fixed step-length baseline does not account for differences in individual gait, walking speed, or activity state[cite: 5]. The moving-average operation provides smoothing but does not by itself remove systematic step-length error[cite: 5].

The current work should therefore be regarded as a lightweight motion-sensing and activity-classification framework rather than a complete indoor positioning solution[cite: 5]. A future positioning implementation can integrate the detected steps, adaptive step-length estimation, compensated heading, and additional inertial sensors to obtain an X-Y trajectory[cite: 5]. Gyroscope-assisted tilt compensation and more adaptive sensor-fusion methods can further improve directional robustness[cite: 5].

#### V. CONCLUSION AND FUTURE WORK

This paper presented a fully offline Edge-AI framework for step counting, heading estimation, distance estimation, and activity classification, 2D/3D mapping using a crossover processor along with accelerometer-magnetometer sensor and a barometer sensor[cite: 5]. The implementation performs sensor calibration, sensor fusion, differential-acceleration-based step detection, fixed-step-length distance estimation with moving-average smoothing, magnetometer-based eight-direction heading estimation, and local Decision Tree activity classification using MicroPython[cite: 5].

Experimental evaluation showed step-counting accuracy above 96% across the tested conditions and an offline activity-classification accuracy of 98.2%[cite: 5]. The deployed Decision Tree also provided a reported classification latency below 0.1 ms with memory overhead below 15 KB of RAM[cite: 5]. The results demonstrate the feasibility of performing lightweight motion analytics and TinyML inference locally on a Cortex-M7 embedded processor without cloud dependence[cite: 5].

Future work will focus on adaptive step-length estimation, gyroscope-assisted tilt compensation, improved magnetometer calibration, and multi-sensor fusion for more accurate GPS-denied indoor trajectory and position estimation[cite: 5].

#### REFERENCES

- [1] W. Xu et al., "Edge Intelligence for Smart Sensing: A Survey," *IEEE Internet of Things Journal*, vol. 8, no. 7, pp. 5200–5219, 2021[cite: 5].
- [2] NXP Semiconductors, "i.MX RT1064 Processor Reference Manual," Rev. 2, 2020[cite: 5].
- [3] STMicroelectronics, "LSM303DLHC Ultra compact high performance ecompass module," Datasheet, 2013[cite: 5].
- [4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011[cite: 5].
- [5] A. Rogers et al., "Zero-Cloud Edge Computing for Embedded Sensors," *IEEE Transactions on Embedded Computing Systems*, vol. 19, no. 4, pp. 310–322, 2022[cite: 5].